CSS Position - Complete Beginner Notes

This guide explains CSS `position` in simple English.
Read it slowly. Try every example in your own `position.html` file.

---

1. What is CSS Position?

Every element on a web page has a place.
By default, elements follow a simple rule called **normal flow**.

Think of normal flow like people standing in a line:

- Block elements (like `div`, `p`, `h1`) stand one below another.
- Inline elements (like `span`, `a`) stand next to each other.

`position` lets you **break this line** and place an element somewhere else.

There are five main values:

```
position: static;   /* default */
position: relative;
position: absolute;
position: fixed;
position: sticky;
```

---

2. The Most Important Rule

Offset properties are:

```
top
right
bottom
left
inset
```

These properties **only work** when the element is positioned.

Positioned means: `position` is **not** `static`.

```
/* This does nothing */
.box {
  top: 50px;
}

/* This also does nothing, because static is the default */
.box {
  position: static;
  top: 50px;
}

/* This works */
.box {
  position: relative;
  top: 50px;
}
```

This is the number one beginner mistake.
If `top`, `left`, or `z-index` is not working, first check the `position` value.

---

3. Normal Flow (the default behaviour)

Before learning position, remember how normal flow works.

```
<div class="box">1</div>
<div class="box">2</div>
<div class="box">3</div>
```

```
.box {
  height: 80px;
  background: lightblue;
  margin-bottom: 10px;
}
```

Result: three boxes, one below another.

- Every box takes its own space.
- Nothing overlaps.
- The page grows as content grows.

When you change `position`, you change this behaviour.
So always ask: "Do I want this element to stay in the flow, or leave the flow?"

---

4. All Values at a Glance

Value	Stays in normal flow?	Moves relative to	Scrolls with page?
`static`	Yes	Nothing (offsets are ignored)	Yes
`relative`	Yes	Its own normal position	Yes
`absolute`	No	Nearest positioned ancestor	Yes
`fixed`	No	The browser viewport (screen)	No
`sticky`	Yes, until a limit	Nearest scrolling container	Until the limit

---

5. Offset Properties Explained

Offsets tell the element **where to move**.

```
top: 20px;     /* move down from the top edge */
right: 20px;   /* move left from the right edge */
bottom: 20px;  /* move up from the bottom edge */
left: 20px;    /* move right from the left edge */
```

Shorthand: `inset`

```
inset: 20px;              /* all four sides */
inset: 10px 20px;         /* top/bottom 10px, left/right 20px */
inset: 10px 20px 30px;    /* top 10px, left/right 20px, bottom 30px */
inset: 10px 20px 30px 40px; /* top right bottom left */
```

`auto`

```
top: auto;
```

`auto` means: "keep the normal position on this side."
This is the default value.

Percentages

Percentages are measured from the **containing block** (explained below).

- `top: 50%` and `bottom: 50%` use the **height** of the containing block.
- `left: 50%` and `right: 50%` use the **width** of the containing block.

This surprises many beginners.
`top: 50%` does not mean "50% of the element's own height."

---

6. Containing Block (Very Important)

A containing block is the **reference box**.
It answers this question: "Moved relative to what?"

`position`	**Containing block**
`static`	The normal parent block

`relative`	The element's own normal position
`absolute`	Nearest ancestor whose `position` is not `static`
`fixed`	The viewport, unless an ancestor has `transform`, `filter`, etc.
`sticky`	Nearest scrolling ancestor

Simple example:

```
<div class="parent">
  <div class="child">I am absolute</div>
</div>
```

```
.parent {
  position: relative;
}

.child {
  position: absolute;
  top: 0;
  right: 0;
}
```

The child sticks to the **top-right corner of the parent**, not the page.
Why? Because the parent is `relative`, so it becomes the reference.

If you remove `position: relative` from the parent, the child will jump to the top-right corner of the page.

---

7. `position: static`

This is the default value.
The element follows normal flow.
Offsets do nothing.
`z-index` does nothing.

```
.box {
  position: static;
}
```

When to use

- Almost all normal content.
- You do not need to write it. It is already the default.

When it does not work

- You cannot move it with `top`, `left`, `bottom`, `right`.
- You cannot control its stacking order with `z-index`.

Common mistake

```
.item {
  z-index: 10;   /* silently ignored */
}
```

The developer thinks `z-index` is broken.
It is not. The element is still `static`.

Exception: flex items and grid items can use `z-index` even when they are `static`. Keep this in mind, but for normal elements you need position.

---

8. `position: relative`

The element **stays in normal flow** and keeps its space.
Then it is moved visually using offsets.

```
.box {
  position: relative;
  top: 20px;
  left: 30px;
}
```

- `top: 20px` moves it **down** by 20px.
- `left: 30px` moves it **right** by 30px.

The space it originally occupied is still reserved.
So other elements do not move.
This can cause overlap. That is normal.

What it is really used for

Most of the time, `relative` is used to create a reference for an `absolute` child.

```
.card {
  position: relative;
}

.badge {
  position: absolute;
  top: 10px;
  right: 10px;
}
```

This is called the **relative parent, absolute child** pattern.
You will use it again and again.

Good things about `relative`

- Keeps the element in the flow.
- Easy to nudge something a few pixels.
- Gives you a reference for absolute children.
- Makes `z-index` work.

Mistakes to avoid

- Do not use `relative` to build whole page layouts. Use flexbox or grid.
- Do not expect other elements to move. They will not.
- Do not forget it on the parent when you use `absolute` children.

---

9. `position: absolute`

The element is **removed from normal flow**.
Other elements act as if it does not exist.

It is placed relative to its nearest **positioned ancestor**.
If there is no positioned ancestor, it is placed relative to the page itself.

```
.child {
  position: absolute;
  top: 0;
  right: 0;
}
```

Key behaviours

1. **No space is reserved.** The parent can collapse in height if all its children are absolute.
2. **Width shrinks to fit content** unless you set a width.
3. **Overlaps other content** easily. Use `z-index` when needed.

Common use cases

- Badge on a card
- Close (x) button on a modal
- Tooltip
- Dropdown menu
- Text on top of an image
- A small label at a corner

Example: Badge on a card

```
<div class="card">
  <span class="badge">New</span>
  <h3>Product</h3>
</div>
```

```
.card {
  position: relative;
  padding: 20px;
}

.badge {
  position: absolute;
  top: 8px;
  right: 8px;
  background: red;
  color: white;
  padding: 4px 8px;
}
```

Fill the parent

```
.overlay {
  position: absolute;
  inset: 0;   /* top: 0; right: 0; bottom: 0; left: 0; */
}
```

If width and height are `auto`, `inset: 0` makes the element fill the parent.

Centre with absolute (older trick)

```
.center {
  position: absolute;
  inset: 0;
  width: 200px;
  height: 120px;
  margin: auto;
}
```

This works, but flexbox and grid are usually easier today.

Mistakes to avoid

1. **Forgetting `position: relative` on the parent.**
   The child runs away to the page corner.
2. **Parent height becomes zero.**
   Absolute children do not give height to the parent.
   Fix: give the parent a `min-height`, padding, or keep one normal child.
3. **Clipping by `overflow: hidden`.**
   If the parent is the containing block and has `overflow: hidden`,
   the absolute child can be cut off.
4. **Thinking absolute is centered by default.**
   It is not. Without offsets it stays at its static position.
5. **Using absolute for the main layout.**
   This breaks easily on different screen sizes.
   Use flexbox or grid for layout.

---

10. `position: fixed`

The element is removed from normal flow.
It is positioned relative to the **browser viewport** (the screen).
It does **not** move when you scroll.

```
.navbar {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
}
```

Common use cases

- Fixed header or navbar
- Floating chat button
- Back-to-top button
- Modal background
- Cookie banner

The transform trap (very common bug)

If any ancestor has one of these properties:

```
transform
filter
perspective
backdrop-filter
will-change
contain: paint
```

then that ancestor becomes the containing block for `fixed` children.
The element will behave like `absolute` inside that ancestor.
It will not stay fixed to the screen.

```
/* Bug example */
.wrapper {
  transform: translateZ(0);
}

.fixed-box {
  position: fixed;
  top: 0;
}
```

Here `.fixed-box` stays inside `.wrapper`.
It will scroll away with the wrapper.
This confuses many developers. Check the ancestors when fixed "stops working".

Mobile height issue

`100vh` can be unreliable on mobile because of the browser address bar.
Use `100dvh` or safe area values.

```
.footer {
  position: fixed;
  bottom: 0;
  padding-bottom: env(safe-area-inset-bottom);
}
```

Other mistakes

- Content hidden behind the fixed bar.
  Add `padding-top` or `padding-bottom` to the body.
- Many fixed elements at the same `z-index`.
  DOM order decides who is on top. Set clear `z-index` values.
- `offsetParent` is `null` for fixed elements in JavaScript.
  This is expected, not a bug.

---

11. `position: sticky`

`sticky` is a mix of `relative` and `fixed`.

- It behaves like `relative` until it reaches a limit.
- Then it sticks like `fixed`.
- It only sticks **inside its parent**.
- It needs at least one offset, like `top: 0`.

```
.header {
  position: sticky;
  top: 0;
}
```

As you scroll down, the header stays at the top of the screen.
When its parent ends, the header leaves with the parent.

When sticky does not work

This is the most complained-about property. Check these one by one.

1. **No offset given.**

```
/* Does nothing */
.header {
  position: sticky;
}
```

Fix:

```
.header {
  position: sticky;
  top: 0;
}
```

2. **An ancestor has `overflow: hidden`, `auto`, or `scroll`.**

```
/* Sticky breaks */
.parent {
  overflow: hidden;
}
```

The sticky element now sticks inside that parent, not the page.
If that parent does not scroll, nothing happens.

3. **The parent has no extra height.**

The element can only stick while its parent has space.
If the parent is exactly the same height as the sticky element,
there is no room to stick.

4. **It is a flex or grid item with `align-items: stretch`.**

The item becomes full height, so it cannot move inside the parent.

Fix:

```
.sidebar {
  position: sticky;
  top: 20px;
  align-self: flex-start;
}
```

5. **The threshold is already passed.**

`top: 0` only sticks when you scroll past the element's normal position.
If the element is already at the top, it may look like nothing happens.

6. **Missing `z-index`.**

The sticky element can be covered by other content while scrolling.
Add a `z-index`.

```
.header {
  position: sticky;
  top: 0;
  z-index: 10;
}
```

Sticky sidebar example

```
<div class="layout">
  <aside class="sidebar">Sidebar</aside>
  <main class="content">Long content...</main>
</div>
```

```
.layout {
  display: flex;
  align-items: flex-start; /* important */
}

.sidebar {
  position: sticky;
  top: 20px;
  width: 220px;
}
```

Mistakes to avoid

- Thinking sticky is the same as fixed.
  Sticky follows the parent and scrolls away at the end.
- Forgetting the offset.
- Leaving `overflow: hidden` on a parent.
- Forgetting `z-index`.

12. `z-index` and Stacking Context

`z-index` decides who is **on top** when elements overlap.

```
.box-a {
  position: absolute;
  z-index: 1;
}

.box-b {
  position: absolute;
  z-index: 2;
}
```

Higher number means closer to the viewer.
Same number means the element later in the HTML is on top.

When does `z-index` work?

- On positioned elements (`relative` , `absolute` , `fixed` , `sticky`).
- On flex items and grid items, even if they are `static` .

It does not work on a normal `static` block element.

Stacking context

`z-index` is not a simple global number.
Elements are grouped into **stacking contexts**.
A new stacking context is created by:

- `position` plus `z-index` other than `auto`
- `opacity` less than `1`
- `transform` , `filter` , `perspective` , `backdrop-filter`
- `will-change`
- `isolation: isolate`
- `mix-blend-mode`
- `contain: paint` or `contain: layout`

Think of it like rooms inside a building.
A child with `z-index: 9999` inside Room A cannot go above Room B
if Room A itself is below Room B.

```
.room-a {
  position: relative;
  z-index: 1;
}

.room-b {
  position: relative;
  z-index: 2;
}

.room-a .child {
  position: absolute;
  z-index: 9999; /* still below room-b */
}
```

This is called the **z-index war**.
The fix is to understand the parent contexts, not to keep increasing numbers.

Other `z-index` mistakes

- Giving a huge number like `99999` and hoping it works.
- Using negative `z-index` and wondering why the element hides behind the parent.
- Forgetting that a transformed parent creates a stacking context.
- Assuming DOM order always wins. It only wins inside the same stacking context.

---

13. Position vs Flexbox vs Grid

`position` is not always the best tool for layout.

Use `position` for:

- Overlays
- Badges and labels

- Tooltips and dropdowns
- Fixed headers and floating buttons
- Sticky section titles

Use **flexbox** for:

- Rows and columns of items
- Centering
- Spacing between items
- Navbar links

Use **grid** for:

- Page layouts
- Galleries
- Two-dimensional layouts

A good rule:
`position` is for **overlap and special placement**.
Flexbox and grid are for **layout**.

---

14. Quick Recipes

Badge on a card

```
.card {
  position: relative;
}

.badge {
  position: absolute;
  top: 8px;
  right: 8px;
}
```

Full-screen overlay

```
.overlay {
  position: fixed;
  inset: 0;
  background: rgba(0, 0, 0, 0.5);
}
```

Sticky header

```
.header {
  position: sticky;
  top: 0;
  z-index: 10;
}
```

Tooltip

```
.trigger {
  position: relative;
}

.tooltip {
  position: absolute;
  bottom: 100%;
  left: 50%;
  transform: translateX(-50%);
}
```

Centre an element in the page

```
.center {
  position: fixed;
  inset: 0;
  display: flex;
  align-items: center;
  justify-content: center;
}
```

---

15. Practice With Your Files

Your project already has `position.html` with `#container` and four boxes.

Try these exercises one by one.

Exercise 1 - Static

```
#box1 {
  position: static;
  top: 100px;
}
```

Question: Does it move?
Answer: No. `static` ignores offsets.

Exercise 2 - Relative

```
#box2 {
  position: relative;
  top: 20px;
  left: 40px;
}
```

Notice: box 2 moves, but the space it had is still there.
Other boxes do not move.

Exercise 3 - Absolute

```
#container {
  position: relative;
}

#box3 {
  position: absolute;
  top: 0;
  right: 0;
}
```

Notice: box 3 leaves the flow and sticks to the container corner.

Then remove `position: relative` from `#container`.
Notice: box 3 jumps to the page corner.
This is the containing block lesson.

Exercise 4 - Fixed

```
#box1 {
  position: fixed;
  bottom: 20px;
  right: 20px;
}
```

Scroll the page. The box stays on the screen.

Exercise 5 - Sticky

Add more content so the page is tall.

```
#box2 {
  position: sticky;
  top: 0;
}
```

Scroll and watch box 2 stick to the top.
Then add `overflow: hidden` on `#container`.
Watch sticky break. This is the overflow trap.

Exercise 6 - z-index

```
#box1 {
  position: absolute;
  z-index: 2;
}

#box2 {
  position: absolute;
  z-index: 1;
}
```

Move the boxes so they overlap. See which one is on top.
Then swap the numbers.

---

16. Common Mistakes Checklist

- [ ] I used `top` or `left` without setting `position`.
- [ ] I used `z-index` on a `static` element.
- [ ] I used `absolute` without `position: relative` on the parent.
- [ ] My parent has zero height because all children are `absolute`.
- [ ] My `absolute` child is clipped by `overflow: hidden`.
- [ ] My `fixed` element is inside a `transform` ancestor.
- [ ] My `fixed` element covers content, and I forgot padding.
- [ ] My `sticky` element has no `top` or `bottom`.
- [ ] My `sticky` element has an `overflow` ancestor.
- [ ] My `sticky` parent is too short.
- [ ] My `sticky` flex item is stretched to full height.
- [ ] I set `z-index: 9999` but the parent context is lower.
- [ ] I expected percentage offsets to use the element's own size.
- [ ] I built the whole layout with `position` instead of flexbox or grid.

---

17. Cheatsheet

I want to...	Use
Keep everything normal	`static`
Make a parent a reference	`relative`
Put a badge in a corner	`relative` parent + `absolute` child
Keep something on screen while scrolling	`fixed`
Stick something until its section ends	`sticky` + `top`
Fill the whole parent	`absolute` + `inset: 0`
Control overlap	`z-index` (check stacking context)
Build a layout	flexbox or grid

---

18. One-Line Summary

`position` takes an element out of its normal place.
Offsets tell it where to go.
The containing block decides what it is measured from.
And `z-index` plus stacking context decide who is on top.

---

19. Quick Self Test

1. Which `position` value ignores `top` and `left`?
2. Which value keeps the element in the flow but still lets you move it?
3. Which value is measured from the viewport?
4. What does `inset: 0` do on an absolute element?
5. Why does `sticky` sometimes stop working?
6. Why does `z-index: 9999` sometimes not work?
7. What does an absolute child need from its parent?

Answers:

1. `static`
2. `relative`
3. `fixed`
4. It fills the containing block if width and height are auto.
5. Often because of an `overflow` ancestor, no offset, or a short parent.
6. Because it is inside a lower stacking context.
7. A parent with `position: relative` (or any non-static position).